reSTITCHing: Refining STITCH

From Single-Run Illustrations to N=3 Evidence and Cost-Controlled Semantic Escalation

Karthikeya Patana Yashwant Rajesh Aathira Sunil Durga Saranyu
Sri Parameswaran Kamesh Chandrasekar Paresh Saxena Soumya Joshi

Abstract

The conference version of STITCH characterised its deterministic semantic interpreter through four outcome regimes (enabling, protective, harmful, and irrelevant), each demonstrated by a single module and model pair evaluated once. That evidence was illustrative, and the paper said so. This extension replaces single-run illustration with N=3 replication on an external benchmark (RTLLM v2.1, 42 modules) across a six-model capability ladder spanning 4B to 32B parameters, and reports what survives it. Two interpreter illustrations replicate cleanly and anchor the mechanism. A mechanistic input/output reachability asymmetry explains *which* defect classes the interpreter can act on, and lets us predict where a tool-augmented repair loop can help before any experiment is run. A conditional-escalation architecture delivers the interpreter's benefit at 35 to 45% lower tool-invocation cost on the in-house benchmark and at tier-specific rates up to 75% at RTLLM scale. The central finding is that firing a repair interpreter on *every* iteration, the conference design, measurably lowers pass rate on small models (a four-point drop on 4B to 8B), while conditional escalation avoids that harm with a structural worst-case guarantee of matching baseline. On the larger models tested that harmful regime is absent, and escalation matches baseline within sampling noise while strengthening its tool-cost advantage. Escalation is therefore the capability-safe choice across the whole ladder: it protects weak models from the harm of indiscriminate intervention, and it costs strong models nothing where the intervention is not needed. A controlled fine-tuning twin comparison (RTLCoder-Deepseek-v1.1 versus its un-fine-tuned base) shows no net fine-tuning advantage at confirmatory scale, a negative result on an external benchmark, though the fine-tuned model does hold a clear edge on signal-processing modules. We harden the evaluation harness against three silent pass-detection defects that had been quietly perturbing measurements, replacing ad-hoc string checks with a structural benchmark-aware interface, and report the defects as a methodological caution for the subfield. The contribution is not that STITCH works in more places, but that we now know *when* it works, *how much* it helps, *at what cost*, and *for which models*, on replicated, tier-decomposed evidence rather than single-draw pooled averages.

1 Why This Extension Exists: N=1 Is Not Enough

The conference paper's table of interpreter outcomes was built on *single runs*: one module, one model, one draw per condition. Its own Limitations section is explicit that the reported percentages are point estimates rather than measured effects. For a repair mechanism evaluated on a stochastic generator, that gap matters more than it first appears. A model at low temperature with no fixed seed can pass one draw and fail the next from sampling alone, so a single-run cell that shows the interpreter "winning" cannot, on its own, distinguish a real mechanism from sampling variance.

Replication mostly confirms the conference results, but the cases it does not confirm are exactly the ones that would have become false claims at N=1. This extension re-runs the mechanism under replication and reorganises the evidence around a single discipline: no interpreter claim is made

without $N \geq 3$ replication and a `semantic_fired` check reported in the same cell it explains. We hold this discipline even when it retires a result we would have liked to keep, including, as Section 8 records, a pooled improvement on large models that the replicate-level trace shows to be sampling noise rather than a fired mechanism. Section 2 presents the cases that survive this bar. Section 3 presents what replication retired, because the retirements define the mechanism's scope as sharply as the successes define its power. Section 4 introduces the cost-controlled escalation architecture. Section 5 reports the harness corrections as a methodological caution. Section 6 describes the full-scale RTLLM study. Section 7 presents the confirmatory fine-tuning twin comparison. Section 8, the core new contribution, is the tier-decomposed and capability-conditioned evidence for when STITCH's escalation architecture acts, when it correctly stays out, and what it can and cannot claim on each part of the capability ladder.

2 Where STITCH Works: Replicated Cases

We report cases where STITCH's mechanism produces a genuine, replicated benefit. Two are the interpreter proper on proven illustrations, and one is a mechanistic reachability result that generalises them. Every case here clears the $N=3$ bar and, where the interpreter is involved, a `semantic_fired` check.

2.1 Two Proven Interpreter Illustrations

The gated-input `UNUSEDSIGNAL` mechanism, the conference paper's cleanest illustration, replicates 3/3 on two modules and two models (Table 1). In `decoder_3to8` and `mux4_oe` the model omits a declared gating port on iteration one, Verilator fires `UNUSEDSIGNAL`, and the interpreter's targeted directive resolves the module in two iterations against a six-iteration failure for both controls. In every cell the `raw_adv` control tracks `baseline` exactly, isolating the effect to *interpretation*, not tool presence.

Table 1: Two proven illustrations, $N=3$ at $T=0.1$. `raw_adv` matches `baseline` in every cell, isolating the effect to interpretation.

Module	Model	baseline	raw_adv	sem_adv
decoder_3to8	gemma3:4b	FAIL — 6	FAIL — 6	**PASS — 2**
decoder_3to8	llama3.1	PASS — 2	PASS — 2	PASS — 2
mux4_oe	gemma3:4b	FAIL — 6	FAIL — 6	**PASS — 2**
mux4_oe	llama3.1	PASS — 4	PASS — 4	PASS — 2

On the enabling cells the interpreter also cuts token cost sharply, roughly 5 to 7× against both controls, because it converts a six-iteration failure into a two-iteration pass. These two illustrations are deliberately the same mechanism (a gating input, `enable` or `oe`, that the model declares but never reads) on two different toy modules. That similarity is not an oversight: several other candidate modules were built to elicit the same defect and discarded because they failed to isolate it, either because the specification accidentally handed the model the answer, or because the resulting defect was a missing default that the system prompt already forces the model to supply. The two survivors are the cases where the gated-input defect is produced cleanly and nothing else is confounded.

These are small modules, and a fair objection is that decoders and gated multiplexers are primitive. The response is that primitives compose. A forgotten output-enable on a multiplexer

that is instantiated two hundred times in a larger design is two hundred latent defects, and a unit-level repair that reliably fixes the primitive is what keeps the system-level design correct. The interpreter's value is not that it repairs toy modules, but that it repairs a defect *class* that recurs at every level of a hierarchy.

2.2 The Input/Output Reachability Asymmetry

A mechanistic result explains *which* defect classes the interpreter can reach, and it is the section most likely to transfer to repair mechanisms beyond STITCH. A forgotten gating *input* reliably produces the target defect and reaches the interpreter. A forgotten *output* does not, because the model assigns declared outputs unprompted on iteration one (Table 2).

Table 2: Forgotten inputs are reachable; forgotten outputs are not. `sf` = `semantic_fired`. The interpreter fires only where the model actually produces the target defect. These are hand-built diagnostic illustrations on gemma3:4b, not RTLLM data.

Target class	**Module**	**Iteration-1 behaviour**	**sf**
Forgotten *input*	decoder_3to8	port declared, never read	1
Forgotten *input*	mux4_oe	`oe` never referenced	1
Forgotten *output*	adder_with_overflow	output assigned unprompted	0
Forgotten *output*	fsm_next_state_error	output assigned on all paths	0

The explanation is a completeness asymmetry in autoregressive generation. Assigning a declared output is a *local* obligation: if the model declares an output and never drives it, the code reads as visibly unfinished, and the model tends to close that gap on its own. Consulting a declared input is a *non-local* obligation: a module that never reads a declared input still compiles and reads as complete, so the omission is only visible by reasoning about the whole module's intended behaviour, and the model does sometimes leave it out.

This directly explains why `semantic_fired` is 0 on the output rows. The interpreter fires only when Verilator emits `UNUSEDSIGNAL`, which occurs when a declared signal is never read. A forgotten output does not produce that diagnostic, and in practice the defect does not occur at all because the model assigns the output unprompted. So `sf`=0 on the output cases does not mean the interpreter failed; it means the target defect never occurred, so there was correctly nothing for the interpreter to fire on, and nothing for the underlying tools to flag either. The two classes are not symmetric: one needs the interpreter, and the other is handled by the generator itself before any tool is consulted.

The practical consequence is a scope claim sharper than the conference paper could make. Reachability is a joint property of the tool and the generator's failure distribution, and it must be measured per class rather than assumed. `UNUSEDSIGNAL` on a gating input is not an arbitrary choice among handled codes; it is the class where tool visibility and real model failure are *known* to overlap. A repair-mechanism designer can use this reasoning to predict, before running any experiment, which defect classes a given tool-plus-generator pairing can and cannot address. Section 8 shows this asymmetry structuring the entire RTLLM-scale result: the modules where escalation avoids harm are those where the model produces a reachable defect signature, and the modules where nothing helps are those where the defect sits outside the tool's visibility.

3 What Replication Retired, and Why It Matters

Three larger-module candidates built to test whether the proven mechanism scales all failed N=3 replication. The failures define the mechanism's boundary and are reported at equal weight with the successes.

Resampling variance can masquerade as a win. `datapath_mux_alu`/llama3.1 passed `sem_adv` in 2 of 3 samples with `semantic_fired`=0 in all three: the interpreter never fired. As a single-run cell this reads as a clean win; it is sampling variance, and N=3 is what exposes it. The same failure mode reappears at scale in Section 8, where a pooled large-model improvement dissolves under a replicate-level trace into draw-to-draw variance with no fired mechanism underneath.

A correct directive does not guarantee recovery. `instr_decode_unit`/gemma3:4b fired `UNUSEDSIGNAL` correctly and still failed, regressing into an unrelated degenerate syntax loop the interpreter has no vocabulary to reach. Reaching a defect and repairing it are different things.

4 Cost-Controlled Semantic Escalation

The conference paper's own harmful and protective regimes point to one structural weakness in firing the interpreter on every iteration: it intervenes when intervention is not needed, and occasionally derails a run that was already converging. The escalation architecture fires the interpreter only once a run is demonstrably stuck. We evaluate three interpreter policies against two controls. The controls are `baseline` (model plus simulator only) and `raw_adv` (the Verilator and Yosys tool layer added but its output fed to the model uninterpreted). The three interpreter policies are:

- `sem_adv` (always-on) fires the interpreter on every iteration. This is the conference paper's design.
- `sem_escalate` begins in the baseline state, tracks consecutive failures carrying the same normalised error signature, and after two identical-signature failures escalates permanently to the interpreter path.
- `sem_escalate_restart` adds a blank-slate rewrite if a run stays stuck even after escalation.

The `raw_adv` control matters because it separates two distinct sources of help. The tool layer itself (compile, lint, and synthesis feedback, in the manner of AutoChip [4]) can rescue a run without any semantic interpretation, and the interpreter's translation layer sits on top of that. Keeping the two separate lets us attribute each effect correctly rather than crediting the interpreter for what raw tool feedback achieved.

The escalation design carries a structural guarantee that always-on interpretation cannot: because escalation begins as baseline and deviates only after baseline has already failed twice on the same signature, on any problem baseline solves, escalation behaves identically to baseline. Its worst case is therefore bounded below by baseline, up to the noise of the two probe iterations. Always-on `sem_adv` has no such floor, and Section 8 shows it falling measurably below baseline on small models. This guarantee is structural, not statistical: it follows from the control flow of the policy rather than from any measured delta, so it needs no confidence interval to defend. The entire value proposition of escalation rests on it: capture the interpreter's benefit where the interpreter helps, while structurally refusing to pay the interpreter's cost, or risk its harm, where it does not.

4.1 The Cost Result on the In-House Benchmark

The defensible headline is efficiency (Table 3). Escalation invokes the heavyweight tools roughly 35 to 45% less often than always-on `sem_adv` and `raw_adv`, while matching or exceeding their pass rate. The two cost axes diverge: escalation reduces *tool invocations* but can *increase* tokens on cells where it escalates and then runs extra iterations. The efficiency claim is scoped to the tool-invocation axis, and both axes are reported.

Table 3: Cost per run, all 255 Phase 1 runs. Efficiency claim scoped to the heavyweight-tool axis; token cost reported and moves the other way.

Condition	**mean tokens/run**	**Verilator+Yosys calls/run**
baseline	4891	0.00
raw_adv	5354	2.61
sem_adv	4883	2.37
sem_escalate	5151	**1.53**
sem_esc_restart	4719	**1.37**

4.2 Replication Collapse in the Benchmark Premise

The Phase 1 benchmark also produced a methodological result. The run set was derived from historical N=1 labels, and re-measured at N=3, 4 of 5 "real-test" and 2 of 3 "enabling" cells do not reproduce the baseline behaviour their category was defined by. This is concrete evidence that single-run RTL-repair benchmarks can encode sampling noise as apparent structure. The regression gate nonetheless holds: both proven illustrations pass 4/4 on strict pass/fail under the new conditions.

5 Harness Hardening: A Methodological Caution

Across this work we found and fixed three silent defects in the shared runner, each verified before and after against golden references. We report them briefly because the pattern, not any single bug, is the point.

Defect A (simulator working directory). The simulation invocation set no working directory, so any testbench doing relative-path file I/O (`$readmemh`) could never resolve its data file, regardless of whether the generated logic was correct. Disqualifying for every file-I/O design; confirmed not to affect the conference benchmark.

Defect B (unclosed code-fence extraction). An unclosed ```` ```verilog ```` fence left the opening marker attached, corrupting otherwise-compilable code. An audit found 58 of 659 generated files (8.8%) affected.

Defect C (pass-string mismatch). The harness checked for the case-sensitive substring `"PASS"` or `"ALL TESTS PASSED"`, but RTLLM's testbenches emit `"Your Design Passed"`, which contains neither. A full re-scan of 64 completed runs found 20 (31%) silently misclassified as failures, and worse, corrupted trajectories: a cell that actually passed on iteration one was told the simulation failed and spent its remaining iterations "fixing" already-correct code under a false signal.

All three are the same underlying fragility: a hardcoded assumption about a benchmark's output format, buried in shared code, that silently produces plausible-looking wrong numbers. This is a caution for the subfield, because a 31% silent misclassification rate is large enough to invert a

published comparison. As a direct consequence, the confirmatory harness routes every pass/fail decision through a single `detect_outcome(·, benchmark)` function backed by a per-benchmark registry, with the benchmark identifier a required argument at every call site and no default fallback: a new benchmark must register its conventions in one place, and forgetting to do so raises an error at import rather than a silent misclassification months later. All confirmatory results in Sections 7 and 8 use this hardened detector, regression-verified against 64 manually-adjudicated cells before any full run was launched.

6 The Full Study: RTLLM at N=3 Confirmatory Scale

To test generalisation beyond the in-house suite we migrated to RTLLM v2.1 [1], an external 50-design benchmark. A batch compatibility screen found 44 of 50 designs compatible with the Icarus flow; the 6 exclusions each have a distinct, documented cause, one of which is a defect in RTLLM's own golden reference. Two further designs (`edge_detect`, `square_wave`) were excluded after their testbenches were found to pass with stuck-at-0 outputs, a weak-oracle defect that would inflate pass rates for weak models. This leaves a 42-module confirmatory pool.

An initial five-module pilot completed only three modules before a thermal stop, and showed no condition differentiation for an instructive reason: the three modules span a ceiling (`counter_12`), a floor (`accu`), and a flat mid-point (`barrel_shifter`), none in the band where STITCH's effect is measurable. The pilot's real yield was a design constraint: STITCH's effect is observable only on modules where baseline sometimes fails but is recoverable, so results must be reported per-module against a difficulty characterisation rather than as a single pooled figure.

Module difficulty tiers are defined empirically from the Twin run's baseline pass rates (Section 7), which cover the widest model spread in this study. For each of the 42 modules, the maximum pass rate across the five Twin model-conditions defines the tier:

- **Ceiling** (6 modules, max pass rate $\geq 80\%$): easy for any capable model.
- **Recoverable band** (17 modules, $20\% \leq$ max $< 80\%$): where any comparison could show discriminating power.
- **Floor** (19 modules, max pass rate $< 20\%$): hard for every model tested.

The STITCH-conditions analyses (Section 8) inherit this tier definition verbatim.

Three confirmatory runs on the pool follow, all using the hardened harness of Section 5. Section 7 presents Set 2, a fine-tuning twin comparison: 5 model-conditions $\times$ 42 modules $\times$ N=3 = 630 runs at T=0.5, baseline-only. Section 8 presents Set 1, STITCH conditions on three small models (4B to 8B): 3 models $\times$ 4 conditions $\times$ 42 modules $\times$ N=3 = 1,512 runs at T=0.1, and Set 3, the same four conditions on three larger models (14B to 32B) restricted to the 17 recoverable-band modules: 3 models $\times$ 4 conditions $\times$ 17 modules $\times$ N=3 = 612 runs at T=0.1. All three completed cleanly, with 0, 6, and 2 transient timeouts respectively.

7 Confirmatory Twin Comparison

7.1 Design

RTLCoder-Deepseek-v1.1 [2, 3] is fine-tuned on a curated Verilog dataset from the same DeepSeek-coder-6.7b base weights [**?**] already in our evaluation. This creates a natural experiment: identical base weights, differing only in fine-tuning, so any performance gap isolates the fine-tuning contribution. We add gemma3:4b [**?**] and llama3.1 [**?**] as ladder context, and run RTLCoder under both its

native stub-completion prompt and the shared harness prompt, making prompt format a measured variable rather than a confound. All models are served locally through Ollama [8].

The scope is 5 model-conditions (RTLCoder native, RTLCoder shared, deepseek-coder shared, gemma3:4b shared, llama3.1 shared) $\times$ 42 modules $\times$ N=3 = 630 pre-registered runs at T=0.5, matched across models for comparability. Condition is fixed at `baseline`: this is a *model* comparison, not a STITCH evaluation. All 630 runs completed with zero errors.

7.2 Fine-Tuning Twin Verdict: A Negative Result With a Real Exception

Contrasting RTLCoder shared against deepseek-coder shared isolates the fine-tuning effect, and this is the section's most notable finding. Across the 17 recoverable-band modules, RTLCoder wins 0, ties 13, and loses 4 (with $|\Delta| \leq 1$ as the tie threshold, matching the N=3 noise floor used in Section 2). Confidence intervals overlap, so this is a lean rather than a knockout, but the direction is honest and runs opposite the fine-tuning claim's own hypothesis: on this external benchmark, at this scale and temperature, fine-tuning RTLCoder on its curated Verilog dataset does not measurably improve its recoverable-band pass rate over the un-fine-tuned base, and on four modules it is worse.

The four losses are not random. Inspecting them individually, they trace to a submodule-hierarchy mismatch, SystemVerilog dialect leakage, a parameter-binding failure, and, on `fixed_point_substractor`, a naming-memorisation artefact in which RTLCoder reproduces RTLLM's folder name over the specification's own stated module name, identically under both prompts. These are fine-tuning side effects, not sampling noise.

Fairness requires reporting where the fine-tuning genuinely helps, and it does. Broken out by functional class, RTLCoder under its native prompt clears every signal-processing module (9/9) and every Memory module (3/3), and holds the Control class where the un-fine-tuned deepseek-coder collapses to 0/6. The net recoverable-band verdict is a wash because the fine-tuning trades ground it loses on general arithmetic for ground it gains on signal-processing and control-heavy designs. Negative results on external benchmarks are worth reporting precisely because a positive claim exists in the literature; we report both the net wash and its functional-class structure rather than collapsing either into a single headline.

7.3 Prompt-Format Effect

Contrasting RTLCoder native against RTLCoder shared isolates prompt format. Across the 17 recoverable-band modules, native wins 3, shared wins 2, and 12 tie. Native shows a modest edge that is inconsistent across modules (Figure 1). We checked the effect and report it as a near-wash rather than claiming a direction.

7.4 Verdict

Two findings survive N=3 confirmatory scale. First, no net fine-tuning advantage for RTLCoder on RTLLM (0 wins, 13 ties, 4 losses), with a real per-class exception: a clear edge on signal-processing and memory modules, offset by losses on general arithmetic. Second, prompt format has a modest, inconsistent effect (3-2-12). Twin's baseline pass rates also serve a structural purpose: they define the difficulty tiers of Section 6 and anchor the capability ladder that the STITCH analysis (Section 8) operates over. Nothing in this section runs STITCH; it is a controlled model comparison.

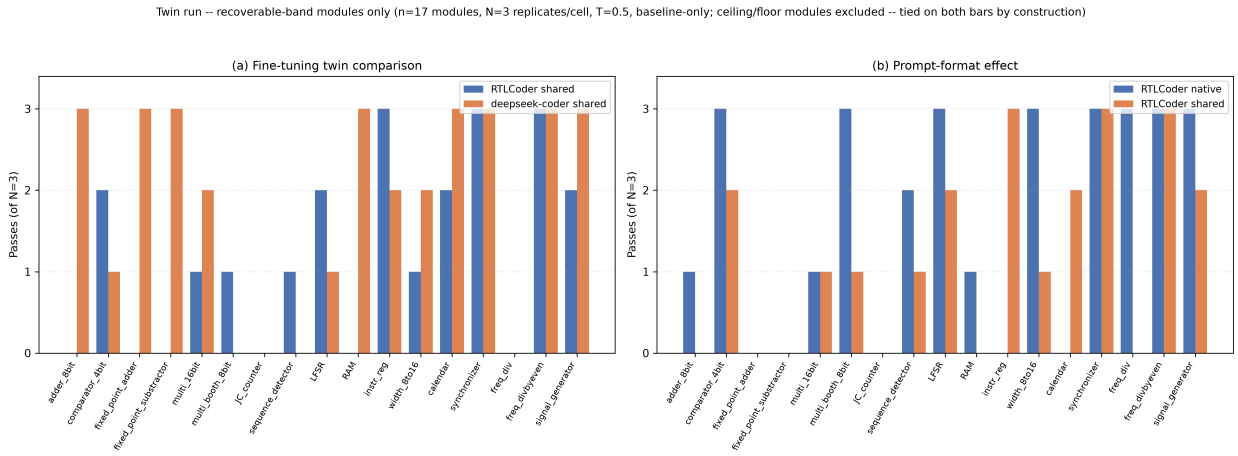


Figure 1: The two named comparisons at RTLLM confirmatory scale. (a) Fine-tuning twin: RTLCoder shared versus deepseek-coder shared, 17 recoverable-band modules; the net verdict is a wash, and several modules (`adder_8bit`, `fixed_point_adder`, `fixed_point_substractor`, `RAM`, `width_8to16`) go against RTLCoder. (b) Prompt-format effect: native versus shared prompt for RTLCoder, same 17 modules; native wins 3, shared wins 2, most tie.

8 STITCH at RTLLM Scale: When the Interpreter Acts and When It Stays Out

The escalation architecture's claim is not that STITCH improves pass rate everywhere. It is that STITCH's interpreter, gated by `sem_escalate`, acts where the model produces reachable defects and stays out where it does not. Two runs quantify this. Set 1 (1,512 runs, three small models, 4B to 8B, all 42 modules) establishes the tier structure and the harmful regime, the paper's headline result. Set 3 (612 runs, three larger models, 14B to 32B, the 17 recoverable-band modules) extends the analysis up the capability ladder and shows the harmful regime is a small-model phenomenon.

8.1 Scope

Set 1: three Ollama models (3 replicates each) across four conditions on all 42 modules, T=0.1. Of 1,512 cells, 6 (0.4%) errored as transient timeouts, all floor-tier; excluded and counted separately. Set 3: `qwen2.5-coder:14b` and `qwen2.5-coder:32b` [**?**], and `gemma3:27b` [**?**], across the same four conditions on the 17 recoverable-band modules, T=0.1 (matched to Set 1 for direct comparability). Of 612 cells, 2 (0.3%) errored as transient timeouts (both `qwen2.5-coder:32b`, one `comparator_4bit` and one `multi_booth_8bit`); excluded and counted. Both runs use the hardened `detect_outcome` registry (Section 5).

8.2 The Tier-Decomposed Pass-Rate Story (Set 1)

Pooled across all 42 modules, the four conditions land within 1.6 percentage points of each other (28.8%, 27.2%, 27.7%, 28.8% for baseline, sem_adv, sem_escalate, sem_escalate_restart). That flatness is not evidence of no effect; it is exactly what the pilot predicted, since pooling ceilings and floors dilutes any recoverable-band signal. Split by tier (Figure 2, Table 4), the structure the pool hides becomes visible.

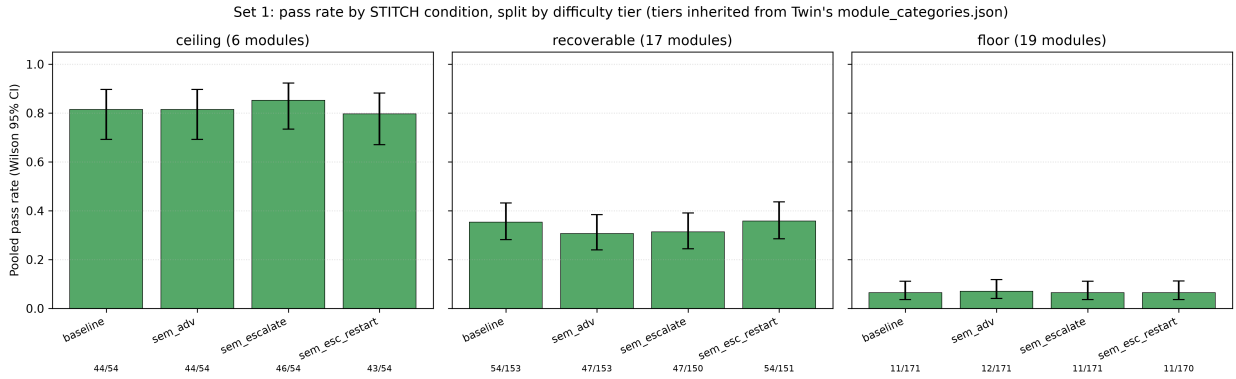


Figure 2: Set 1 pass rate by STITCH condition, split by difficulty tier inherited from the Twin run. Error bars are Wilson 95% CIs. On the recoverable band, `sem_adv` drops below baseline (the harmful regime), and `sem_escalate_restart` recovers to baseline. On ceilings and floors all conditions coincide.

Table 4: Pass counts by tier and STITCH condition, Set 1 (3 small models × N=3).

Tier	**Modules**	**baseline**	**sem_adv**	**sem_esc**	**sem_esc_restart**
Ceiling	6	44/54	44/54	**46/54**	43/54
Recoverable	17	54/153	47/153	47/150	**54/151**
Floor	19	11/171	12/171	11/171	11/170
Pooled	42	109/378 (28.8%)	103/378 (27.2%)	104/375 (27.7%)	108/375 (28.8%)

Three findings, in order of confidence. **(1) The harmful regime replicates at scale.** On the recoverable band, `sem_adv` drops about four points below baseline (30.7% versus 35.3%). This is the exact behaviour the conference paper's harmful regime predicted from a single cell: always-on interpretation intervenes when it should not and occasionally derails a converging run. To our knowledge this effect has not previously been demonstrated at multi-module, replicated scale in RTL repair, and it is the paper's headline: more tool-driven intervention is not free, and on the modules where it matters it measurably lowers pass rate. **(2) Escalation protects the recoverable band from that harm.** `sem_escalate_restart` recovers the four-point drop and matches baseline exactly (54/151 versus baseline's 54/153), the worst-case-equals-baseline guarantee of Section 4 confirmed at scale. **(3) Ceilings and floors behave as designed.** On ceilings all four conditions produce essentially the same pass rate (80 to 85%); on floors all four produce 6 to 7%, the defects there sitting outside the interpreter's reachable class (Section 2.2).

8.3 The Tier-Decomposed Cost Story (Set 1)

The pass-rate story would be a null result if escalation cost the same as `sem_adv` everywhere. It does not (Table 5).

Table 5: Mean heavyweight-tool invocations per run, Set 1, by tier. `baseline` is structurally 0. The reduction column is `sem_escalate` relative to `sem_adv`.

Tier	Modules	sem_adv	sem_esc	Reduction
Ceiling	6	4.02	1.00	**75.1**%
Recoverable band	17	6.74	5.62	16.6%
Floor	19	7.28	6.54	10.1%
Pooled	42	6.60	5.38	18.5%

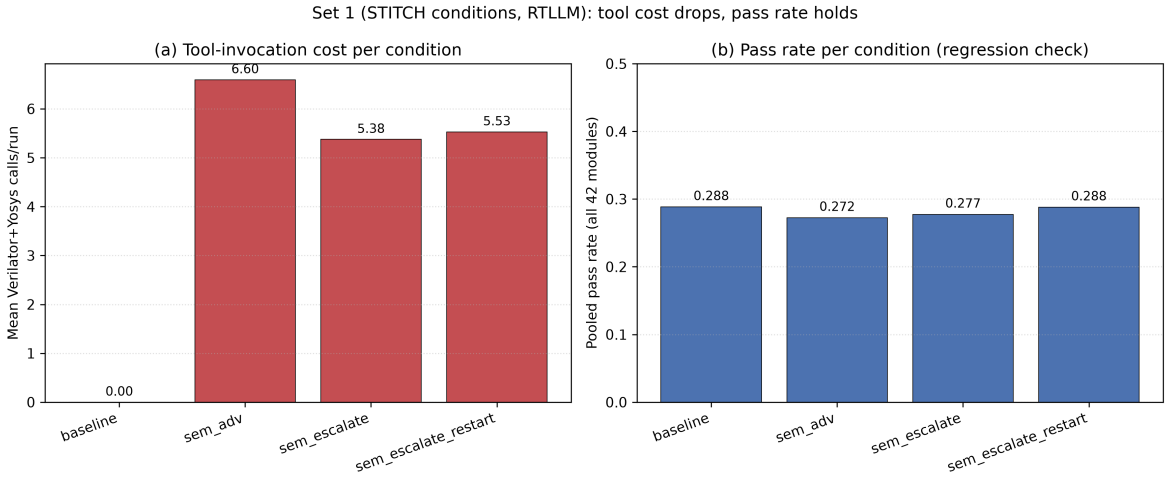


Figure 3: Set 1, 1,512 runs. (a) Mean Verilator+Yosys invocations per run: escalation cuts the heavyweight-tool axis against always-on `sem_adv`, pooled across all 42 modules. (b) Pooled pass rate holds within 1.6 points across all four conditions, the regression check the cost claim requires.

The reduction follows a clean monotonic gradient by tier: 75.1% on ceilings, 16.6% on the recoverable band, and 10.1% on floors. On modules where baseline succeeds, escalation's two-strike

gate almost never fires (the tool call rate drops from 4.02 to 1.00 per run), while pass rate holds. On the recoverable band and floor, escalation does trigger, because runs do get stuck. Escalation's cost discipline concentrates on the modules where tools have the least to offer, not by explicit policy but as a byproduct of the two-strike gate tracking recoverability. The pooled 18.5% is smaller than Phase 1's 35 to 45% because RTLLM's mix is dominated by ceilings and floors; the tier-level gradient is what is stable. The token axis moves the other way, as on the in-house benchmark: escalation spends more tokens per run because escalated runs execute additional iterations. The efficiency claim is scoped throughout to the heavyweight-tool axis.

8.4 Up the Capability Ladder: The Harmful Regime is a Small-Model Phenomenon (Set 3)

Set 1 shows escalation *protecting* small models from the harmful regime. Set 3 asks what happens as capability rises, running the same four conditions on three larger models (14B to 32B) over the same 17 recoverable-band modules at the same temperature. The finding is that the harmful regime largely disappears: at the group level the mean `sem_adv` minus baseline delta rises from -4.6 points on the small models (Set 1) to -0.65 points on the large models (Set 3, Figure 4). Always-on interpretation stops measurably hurting once the models are strong enough to get stuck less often.

Pooled across the three larger models (Table 6), all four conditions land within about three points of each other, with the two escalation policies nominally highest (`sem_escalate` 58.8%, `sem_escalate_restart` 58.3%) and `baseline` at 56.2%. We do not claim this as a win. A replicate-level trace of the 153 `sem_escalate` runs shows why: 88 passed without ever escalating (the model succeeded in the first two iterations, tools never engaged), 65 escalated, and of those only 2 eventually passed, neither with the semantic interpreter having fired (`semantic_fired`=0 throughout, both rescued by raw Verilator and Yosys feedback plus the extra iteration budget). Across all 153 runs, not a single passing replicate had the interpreter fire. The net pooled improvement over baseline is four cells out of 153 flipping by one replicate each, three cells flipping the other way, and every confidence interval crossing zero. This is the same resampling-variance pattern Section 3 describes, now at scale: independently drawn N=3 samples differ by exactly the amount sampling noise predicts, and there is no fired mechanism underneath. The honest reading is that on large models escalation does no harm and matches baseline within noise, which is precisely what the structural guarantee of Section 4 promises and all it promises.

Table 6: Set 3 pooled recoverable-band pass rate, three larger models (14B to 32B) $\times$ N=3, T=0.1. All four conditions coincide within noise; the escalation lead is not a fired-mechanism effect (see text).

Condition	Passes / n	Rate
baseline	86/153	56.2%
sem_adv	85/153	55.6%
sem_escalate	90/153	58.8%
sem_esc_restart	88/151	58.3%

The per-model picture (Table 7) confirms the harmful regime is model-specific rather than a smooth function of scale. `gemma3:27b` reverses it, with `sem_adv` 7.8 points above its baseline, while `qwen2.5-coder:32b`, the most capable model, still shows a -5.9-point harm comparable to a small model's. Reporting this non-monotonicity is the honest counterpart to not claiming the pooled

lead: the effect that is real (per-model harm on some models) and the effect that is not (a general escalation pass-rate win) are separated rather than averaged into a single misleading number.

Table 7: Set 3 per-model recoverable-band pass rate (n=51 per cell, except `qwen2.5-coder:32b` sem_esc_restart n=49 after two timeouts). The `sem_adv` column shows the harmful regime is model-specific: present on `qwen2.5-coder:32b`, reversed on `gemma3:27b`.

Model	**baseline**	**sem_adv**	**sem_esc**	**sem_esc_restart**
qwen2.5-coder:14b	52.9%	49.0%	56.9%	58.8%
gemma3:27b	37.3%	45.1%	37.3%	33.3%
qwen2.5-coder:32b	78.4%	72.6%	82.4%	83.7%

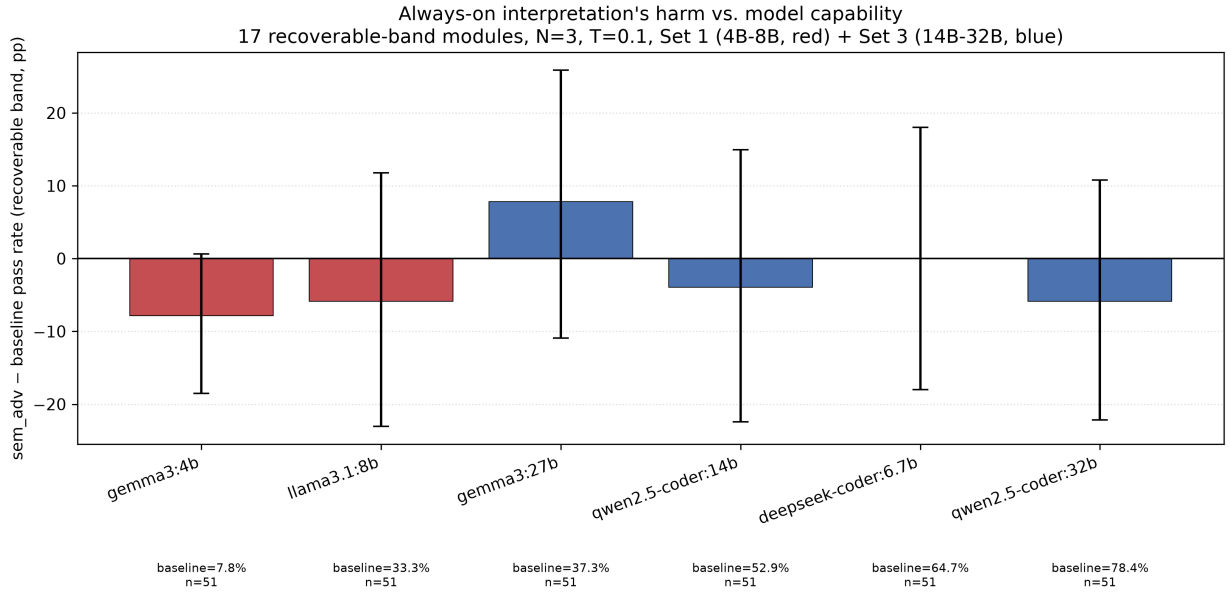


Figure 4: Always-on interpretation's harm (`sem_adv` minus baseline, percentage points on the recoverable band) against model capability, all six models, N=3, T=0.1. Set 1 small models in red, Set 3 large models in blue, ordered by baseline pass rate. The group mean harm shrinks from −4.6 (small) to −0.65 (large), but per-model the effect is not monotonic: `gemma3:27b` reverses it while `qwen2.5-coder:32b` retains it. Wilson 95% CIs, all crossing zero on the large models.

The cost discipline, unlike the pass rate, does hold and strengthen at scale. On the recoverable band, the `sem_adv` to `sem_escalate` tool-invocation reduction is 27.6% for the Set 3 models (5.31 to 3.85 calls per run), against 16.6% for the Set 1 models on the identical module band. Capable models get stuck less often, so escalation triggers its heavyweight path less often, and the cost advantage over always-on interpretation widens. This is a real effect, not a noise artefact: it is a per-run mechanical count of tool invocations, not a pass-rate delta.

The extended capability ladder (Figure 5) places all six models on a single axis spanning 4B to 32B, confirming that the range is well covered and that the strong models sit where expected.

Because Set 3 runs at T=0.1 and the Twin models at T=0.5, the comparison across the two temperatures is ordinal (the ladder ordering) rather than a direct rate match, and the figure marks this.

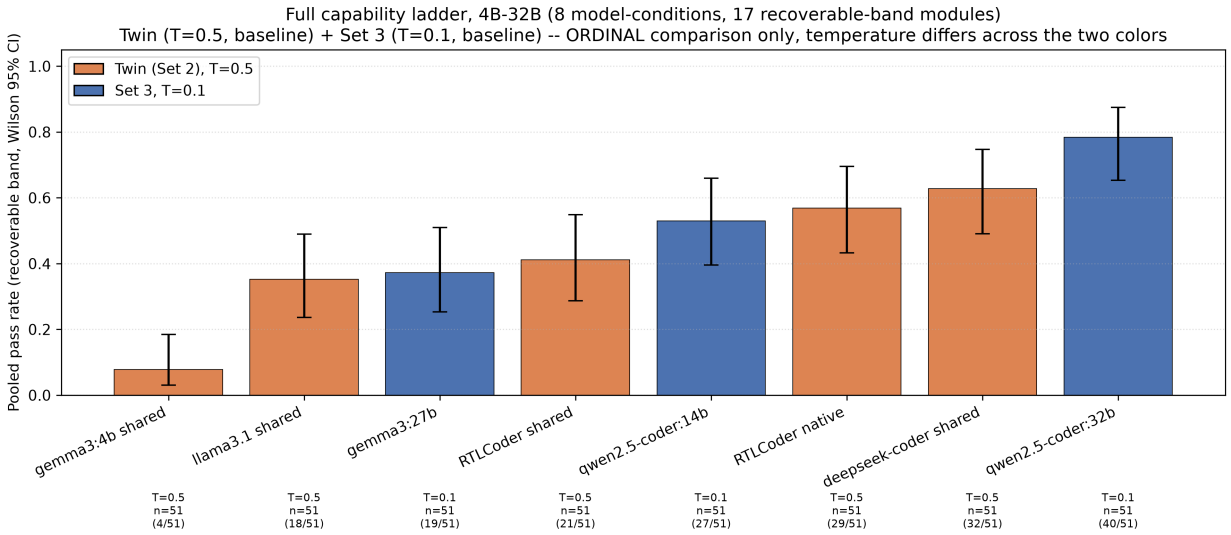


Figure 5: Full capability ladder, 4B to 32B, eight model-conditions on the 17 recoverable-band modules, `baseline`. Twin (Set 2, T=0.5) in orange, Set 3 (T=0.1) in blue; the two colours are an ordinal comparison only, since temperature differs. Wilson 95% CIs, n=51 per bar.

8.5 What This Establishes

Read together with Sections 2 and 4, the two STITCH runs support five claims:

1. The proven-illustration mechanism (Section 2.1) generalises to a per-tier prediction: the interpreter helps where it can reach a real defect and stays silent where it cannot.
2. The harmful regime of always-on interpretation is real, replicated across 17 modules and three small models at N=3, and measurable (about four points below baseline). This is the paper's central positive finding: on small models, firing a repair interpreter indiscriminately measurably lowers pass rate.
3. Escalation avoids that harm on small models, matching baseline on the recoverable band, and on large models matches baseline within noise. The worst-case-equals-baseline guarantee holds across the whole ladder, and it is structural rather than statistical.
4. The harmful regime is a small-model phenomenon: it fades at the group level as capability rises, though per-model it is not monotonic (`gemma3:27b` reverses it, `qwen2.5-coder:32b` retains it). We do not claim a large-model pass-rate win for escalation; the pooled lead is sampling noise with no fired mechanism underneath.
5. The cost result is tier-shaped and strengthens with capability: escalation cuts about three-quarters of heavyweight-tool calls on ceilings and its recoverable-band reduction rises from 16.6%

on small models to 27.6% on large ones. This is a mechanical per-run count, unaffected by the pass-rate noise.

None of this establishes STITCH as a universal pass-rate improvement; the pooled numbers and the replicate-level trace make that explicit. It establishes something narrower and more defensible: always-on interpretation, the conference design, measurably hurts weak models, and conditional escalation is the mechanism that captures the interpreter's benefit where it fires while structurally guaranteeing no harm and lower tool cost everywhere else. On weak models that guarantee is an active rescue from a real harm; on strong models it is a promise kept at no cost. That is a sharper claim than a broad pass-rate win, and it is supported by the largest replicated evaluation of a tool-augmented RTL repair loop we are aware of.

9 Conclusion and Directions

The conference version of STITCH showed the interpreter could help, via four single-run outcome regimes on hand-selected cells. This extension replaces that evidence with N=3 replicated, tier-decomposed evidence across 42 modules and a six-model 4B-to-32B ladder on an external benchmark, and answers four questions the conference paper deferred. *When* does the interpreter help: on the recoverable band, on small models, and only when the model produces a defect from the reachable input class (Section 2.2). *How much*: firing the interpreter every iteration costs small models about four points on the recoverable band, and escalation recovers that loss; on large models the harm is absent and escalation matches baseline within noise. *At what cost*: tool cost concentrates where it can help and is nearly eliminated where it cannot, with the reduction widening from 16.6% to 27.6% as capability rises. *For which models*: escalation protects weak models from a real harm and costs strong models nothing, making it the capability-safe choice across the ladder. The study also contributes a negative result independent of STITCH, that RTLCoder's fine-tuning shows no net advantage over its base on RTLLM's recoverable band (with a real per-class exception on signal-processing and memory designs), and a methodological caution, that silent pass-detection bugs can misclassify a third of runs and invert a comparison.

Directions. Two extensions are natural. First, and more substantial, a defect-class expansion of the reachability asymmetry (Section 2.2): the study's 19 floor-tier modules provide a large corpus of generations that fail under every model, which can be inspected for defect classes the interpreter currently has no vocabulary for, and for each reachable-but-unhandled class the interpreter's directive set can be extended and the affected modules re-measured. This is the path from "STITCH works where it works" to "here is a principled way to grow what it works on," and it is the natural spine of the next paper. Second, a robustness study under imperfect specifications, injecting specification-level defects to measure how the interpreter's behaviour differs between model defects and specification defects. Frontier closed-source models, reachable only by API, are a further point on the capability ladder worth adding once the recoverable-band scope keeps their evaluation cost tractable; the `gemma3:27b` non-monotonicity suggests the harmful regime's dependence on a model's specific post-training, not just its scale, is itself worth a targeted study.

Two commitments carry forward: every interpreter claim remains $N \geq 3$-replicated and `semantic_fired`-gated, and every pass-rate figure is reported against the difficulty tier that determines whether the mechanism could have been measured. Holding that second commitment is what turned an apparent large-model win into a correctly-reported null, and the paper is stronger for it. The conference paper showed STITCH could help; this extension establishes when, how much, at what cost, and for which models, on evidence built to be difficult to argue with.

References

[1] Y. Lu, S. Liu, Q. Zhang, and Z. Xie, "RTLLM: An Open-Source Benchmark for Design RTL Generation with Large Language Model," in *Proc. ASP-DAC*, 2024.

[2] S. Liu, Y. Lu, W. Fang, M. Li, and Z. Xie, "RTLCoder: Outperforming GPT-3.5 in Design RTL Generation with Our Open-Source Dataset and Lightweight Solution," in *Proc. IEEE LAD*, 2024.

[3] HKUST Zhiyao Lab, "RTLCoder-Deepseek-v1.1," Hugging Face model repository, 2024. https://huggingface.co/ishorn5/RTLCoder-Deepseek-v1.1

[4] S. Thakur, J. Blocklove, H. Pearce, B. Tan, S. Garg, and R. Karri, "AutoChip: Automating HDL Generation Using LLM Feedback," in *Proc. DAC*, 2024.

[5] W. Snyder, "Verilator: Fast, Free Verilog HDL Simulator," 2023. https://www.veripool.org/verilator/

[6] C. Wolf, "Yosys Open Synthesis Suite," in *Proc. IWLS*, 2012. https://yosyshq.net/yosys/

[7] S. Williams, "Icarus Verilog," 2023. https://steveicarus.github.io/iverilog/

[8] Ollama, "Ollama: Get up and running with large language models locally," 2024. https://ollama.com

[9] Y. Tsai et al., "RTLFixer: Automatically Fixing RTL Syntax Errors with Large Language Models," *arXiv:2311.16543*, 2023.

[10] M. Liu et al., "VerilogEval: Evaluating Large Language Models for Verilog Code Generation," *arXiv:2309.07544*, 2023.

[11] A. Madaan et al., "Self-Refine: Iterative Refinement with Self-Feedback," in *Proc. NeurIPS*, 2023.

[12] S. Fung, "HDLBits: Verilog Practice Problems," 2023. https://hdlbits.01xz.net

[13] P. Lewis et al., "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks," in *Proc. NeurIPS*, 2020.

[14] BITS-Pilani, Hyderabad Campus, "Institutional High-Performance Computing (HPC) Facility," 2026.

[15] E. B. Wilson, "Probable Inference, the Law of Succession, and Statistical Inference," *J. Amer. Statist. Assoc.*, vol. 22, no. 158, pp. 209–212, 1927.